You just successfully migrated all your code to a Git repository.

You can now add a remote Git repo, and sync your code to the remote repository as follows:

```
$ git remote add origin git@github.com:user/my_remote_repo.git
```

Grepping through files

Searching through source code is a big part of a programmer's life, and *grep* is used extensively for this. Git has an inbuilt *grep* command to search through files tracked by the repository—*git grep text*.

Git describe

The *git describe* command describes the current commit based on the last tagged version. Try *git describe* and you may get something like:

```
1.0-62-g4e975db
```

This auto description can be interpreted as follows:

- *1.0* – The recent tag
- *62* – The *HEAD* is the 62nd commit after the tagged commit 1.0
- *g4e975db* – The first eight characters of the HEAD commit ID.

Integrating with Github

Github is a great Git repository hosting website, which has lots of facilities to collaborate with many developers on a project, including features that help to code, integrate, merge and perform code reviews. Let us go through an example Git workflow along with GitHub integration.

Example workflow

The following is the most common Git workflow that everyone uses daily.

```
$ mkdir myproject # Decided to start a new project
$ git init #Add project as a git repo
# Created few files README, main.c. Now I wanted to add those files
to repo
$ git add README main.c
$ git commit -m "Initial commit"
$ git log # View the log for the commit
#Later I want to change the commit message to a more appropriate
one.
$ git commit --amend
$ git diff # I modified main.c. I would like to see the diff
# Now I would like to add the changes into two commits
$ git add -p main.c #Interactively select few code chunks and add
# Commit the selected changes
$ git commit # vim opens up and we add a commit message such as:
Fix bug-0234 - limit the size of the array
<Blank line>
Description about bug-0234 and fix
#Add rest of the changes to another commit
$ git add main.c
$ git commit
$ git log
# Now to sync my repo to a remote (Github) repository I created.
# Add a remote repo to current repo
git remote add origin https://github.com/t3rm1n4l/projectname.git
$ git push origin master # Push master branch (Default) to remote
repo
$ git branch -a # List branches
```

Now, if you want to work on a feature that takes a lot of time, while some other development goes on in parallel, create a branch for the feature development as shown below:

```
$ git branch feature-x
$ git checkout feature-x
```

To add some changes related to *feature-x* (such as, adding a *TODO.txt*):

```
$ git add TODO.txt
$ git commit -m "Added TODO for feature-x"
```

To go back to the master branch and make a few commits (like adding and changing few files), issue the following commands:

```
$ git add core.c
$ git commit -m "Move core functions to core.c"
$ git add main.c
$ git commit -m "Refactor main.c"
```

If you want to work on *feature-x*, do a *git checkout feature-x*, then add a bunch of files to build the feature, and then use the following code:

```
$ git add feature.c main.c core.c
$ git commit (message as follows)
Add feature-x - logger module for xxx
<Blank line>
More description about the feature
```

Switch back to the master branch to make some more changes:

```
$ git checkout master
$ git add README (commit some changes from README)
$ git commit -m "Improve README"
```

If you then want to merge the *feature-x* I developed into